Assignment 02

Motion planning

Introduction

Motion planning is the set of techniques that calculate collision-free paths for a robot or an autonomous vehicle, going from an initial configuration to a final one based on geometric, dynamic and motion-cost data.
Among the classic global motion planning algorithms are Dijkstra and A*.

Dijkstra Algorithm

The Dijkstra algorithm finds the minimum-weight path between a pair of vertices in a directed, weighted graph. It solves the “single-source shortest-path” problem by creating a table of information on the best known way to reach each vertex (cost, previous vertex) and improving it until the optimal solution is reached. The basic principle of the algorithm is to initialize all nodes with infinite distance except the source, use a priority queue to always select the node with the minimum distance, and update the distances of its neighbors if a better path is found. Although Dijkstra guarantees the optimal path, in large graphs it visits a high number of nodes, resulting in a high computational cost.

A* Algorithm

A* is an informed graph-search algorithm that finds the least-cost path from a start node to a goal node by combining the path cost from the start with an admissible estimate of the remaining cost. It maintains a frontier of partial paths and always expands the path whose total estimated cost $f(n) = g(n) + h(n)$ is smallest. By tuning the heuristic $h(n)$, A* smoothly interpolates between uniform-cost search and greedy best-first search.

The main difference with Dijkstra is that A* “considers not only how far a node is from the start, but also how close it is to the goal.” This allows A* to “focus on the promising nodes at the frontier and find the optimal path faster than Dijkstra.” To highlight the differences between the two, as a testset we chose both very large, densely roaded cities and others less extensive with a simpler urban layout.

Place	Number of nodes	Number of edges
Paris, France	9477	18223
Piedmont, California, USA	352	944
Turin, Piedmont, Italy	12001	25795
Berlin, Germany	28222	73370
Rome, Italy	43573	90267
Barcelona, Spain	8865	16445
Madrid, Spain	31088	61227
London, England	129225	301162
New York City, New York, USA	55247	139290

Code Analysis

The code implements a complete system for comparing path-planning algorithms on real road networks, structured in a logical flow that combines data acquisition, algorithmic processing and advanced visualization. The search for the optimal path is performed in three separate iterations-each

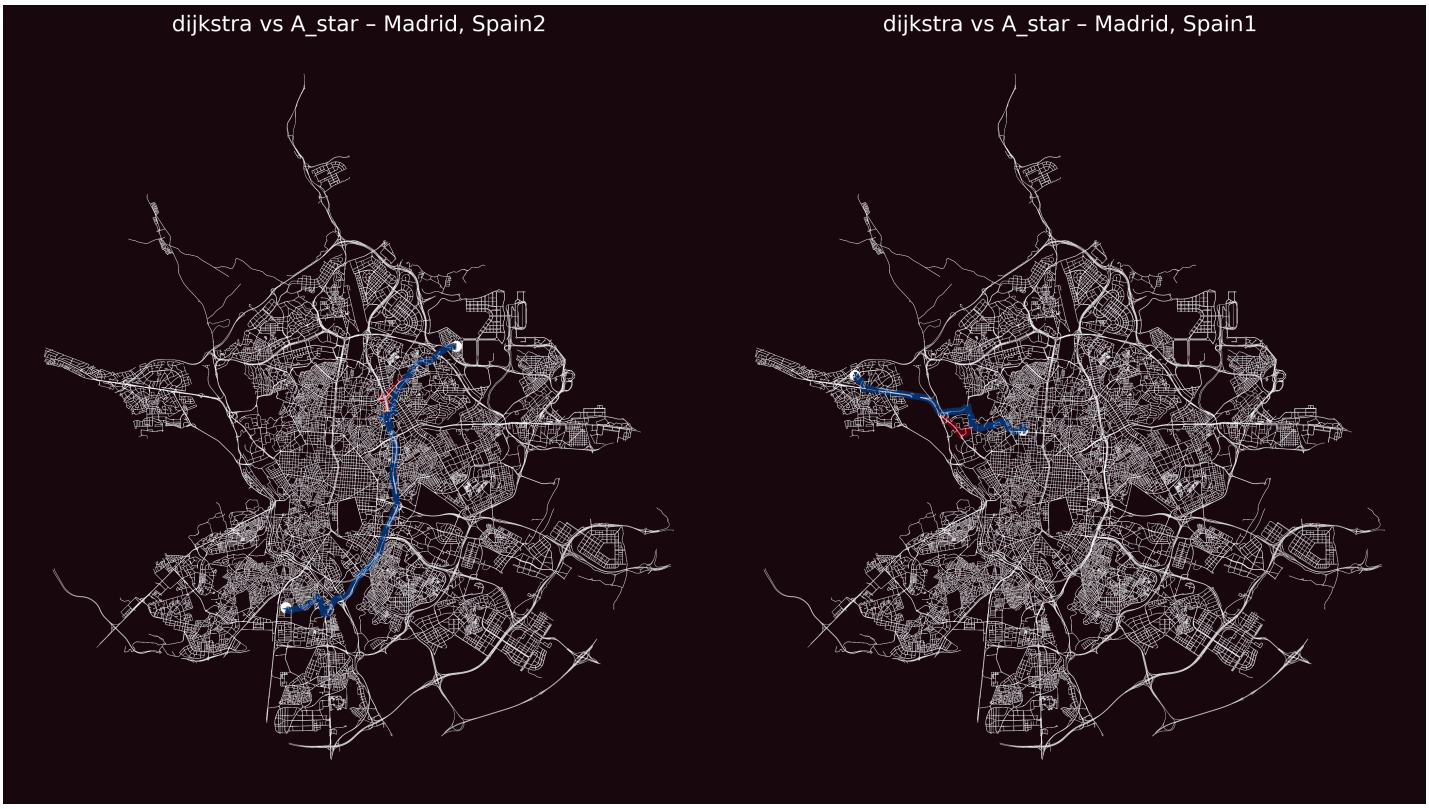
using different start and end points within the same city-and then repeated for each city. The entire process is divided into **four integrated macro-phases.**

1. **Acquisition and preparation:** For each specified location and different algorithms, a detailed road graph is generated using the **ox.graph_from_place()** function via the OSMnx library, which captures the entire network, out of the iteration using different starts and ends because it is the action more time consuming. To not overwrite the path of the two algorithms, there is a graph for each algorithms. Edge weights are calculated as the ratio between physical length and maximum speed, creating a realistic time-cost metric for navigation. This thing will be important also for the definition of the heuristic for A*. The important part is the setting of maximum speed because in some edge the max speed is not a number because they have the **tag OSM** as 'walk'. To support the testing in implementation phase you can use the args **—no-plot** to be faster to obtain the final textual report overcoming the plots of the paths.

```
for place_name in place_names:
    G = ox.graph_from_place(place_name, network_type="drive")
    F = ox.graph_from_place(place_name, network_type="drive")
    number_of_nodes.append(len(G.nodes))
    number_of_edges.append(len(G.edges))
    print("Place:", place_name)
    print("Number of nodes:", len(G.nodes))
    print("Number of edges:", len(G.edges))
    for i in range(3):
        print("Iteration on this city :", i)
        print("\n")
        for edge in G.edges:
            maxspeed = MAX_SPEED
```

2. **Implementation of the algorithm:** The two implemented algorithms-Dijkstra and A*-share a common foundation but diverge strategically in their approach to finding the optimal path.
   Dijkstra and A* use a classic priority queue, exploring nodes in strict order of increasing distance from the source. This approach guarantees path optimality but involves a "concentric-circle" exploration that can be inefficient in large networks.
   A* introduces an innovative element through a geodetic heuristic, computed via the Haversine formula for great-circle distance between nodes and then divided for the max speed. In this way g(n) and h(n) are consistent. The combined evaluation function f(n) = g(n) + h(n) allows the search to be strategically oriented toward the destination, drastically reducing the number of nodes examined. The implementation leverages a dual data structure (open_set and closed_set) to track exploration state, optimizing memory usage.
3. **Visualisation:** During algorithm execution, dynamic styling modifies the appearance of edges in real time to annotate each algorithm's path. Once

the cycle for a given location is complete, heatmaps generate single and comparative visualizations: single maps show the edges for each algorithm, while the comparative map's color overlay (red for Dijkstra, blue for A*) highlights the strategic divergences between the two approaches.



4. **Report generation:** The system logs, for each run, the total number of iterations and the actual path length. The final report combines aggregated statistics and percentage comparisons, highlighting performance patterns through a multi-criteria analysis (distances, steps, node,edge) comparing the algorithms.

```
with open("results.txt", "w") as file:
    for i in range(len(place_names)):
            for j in range(3):
                index = i * 3 + j
                result = (
                    f"Place: {place_names[i]}, Iteration: {j}, "
                    f"Distance {distaces[index]:.2f}km\n"
                    f"Dijkstra: {interations_Dijkstra[index]}, "
                    f"A*: {interations_A_star[index]}\n"
                )

                if interations_A_star[index] != 0:
                    slowdown = (
                        (interations_Dijkstra[index] - interations_A_star[index])
                        * 100
                        / interations_A_star[index]
                    )
                    comparison = f"Dijkstra è {slowdown:.2f}% più lento di A*\n"
                else:
                    comparison = "A* non ha completato, confronto non possibile.\n"

                information = (
                    f"Number of nodes: {number_of_nodes[i]}, "
                    f"Number of edges: {number_of_edges[i]}, "
                    f"Number of edges algorithm: {number_of_edges_algorithm[index]}\n\n"
                )

                print(result.strip())
                print(comparison.strip())
                print(information.strip())
```

Experimental Results

From the comparative images, it is evident that the algorithms converge to the same solutions. This statement is the result of various robustness tests explicitly using two completely different graphs for each algorithm, avoiding buffer and path-overwrite errors.
The A* algorithm consistently outperformed Dijkstra in terms of efficiency.

A comparison over 27 iterations in nine cities shows that Dijkstra requires on average 14,971 iterations versus 3,616 for A*, making it roughly 314% slower (average slowdown per run: 968%). Within the same city the gap varies: in Paris Dijkstra is 161–205% slower, in Piedmont 117–478%, and in Turin 376–617%, depending on start–end orientation. In smaller graphs (e.g. Piedmont with 352 nodes and 944 edges) the gap remains modest, whereas in medium/large ones such as Berlin (28,222 nodes, 73,370 edges) and Rome (43,573 nodes, 90,267 edges) Dijkstra explores almost the entire graph, with extreme slowdowns up to 4,492% and 1,924% respectively, since A* effectively uses its heuristic to prune the search. In very large networks like London (129,225 nodes, 301,162 edges), a near-linear route reduces the slowdown to just 70%, while in New York City (55,247 nodes, 139,290 edges) the gap stays between 272% and 543%, highlighting how network topology and point-to-point geometry heavily influence A*'s efficiency. Overall, A* cuts the iteration count by over 75% compared to Dijkstra.

Place (Iteration)	Distance (km)	Dijkstra	A*	Dijkstra è % più lento di A*	Number of edges algorithm
Paris, France (0)	6.94	5867	2248	160.99%	51
Paris, France (1)	8.58	4995	1651	202.54%	37
Paris, France (2)	7.46	5933	1945	205.04%	83
Piedmont, California, USA (0)	0.74	52	9	477.78%	7
Piedmont, California, USA (1)	2.42	137	50	174.00%	19
Piedmont, California, USA (2)	2.22	271	125	116.80%	19
Turin, Piedmont, Italy (0)	0.81	293	47	523.40%	12
Turin, Piedmont, Italy (1)	13.79	10791	1506	616.53%	154
Turin, Piedmont, Italy (2)	6.14	9715	2043	375.53%	78-79
Berlin, Germany (0)	13.32	9844	715	1276.78%	85
Berlin, Germany (1)	5.69	2734	68	3920.59%	46
Berlin, Germany (2)	5.49	2893	63	4492.06%	38
Rome, Italy (0)	15.10	26698	5141	419.32%	143-147
Rome, Italy (1)	4.06	543	90	503.33%	47
Rome, Italy (2)	14.33	17771	878	1924.03%	96
Barcelona, Spain (0)	6.03	7153	456	1468.64%	50-51
Barcelona, Spain (1)	5.47	3831	168	2180.36%	53-52
Barcelona, Spain (2)	2.41	1080	95	1036.84%	29
Madrid, Spain (0)	9.32	12547	1289	873.39%	109-109
Madrid, Spain (1)	9.73	12193	682	1687.83%	73-77
Madrid, Spain (2)	17.49	25794	1506	1612.75%	121-15
London, England (0)	13.47	34166	7547	352.71%	120
London, England (1)	27.11	64084	37748	69.77%	172
London, England (2)	9.47	29865	7401	303.53%	155
New York City, New York, USA (0)	18.95	37882	8456	347.99%	139
New York City, New York, USA (1)	17.78	32870	8838	271.92%	164
New York City, New York, USA (2)	24.12	44220	6875	543.20%	122

In same cities, as Madrid, the paths are a little different. The discrepancy almost always boils down to one of two factors: the heuristic in A* or the existence of multiple equally-short routes.

First, if A* is driven by a heuristic that isn't strictly admissible (i.e. it sometimes overestimates the remaining distance), it can settle on a path that appears cheapest under its f(n)=g(n)+h(n) ordering even though a truly optimal route exists-Dijkstra, having no heuristic, will then produce the true shortest path.

Second, even with an admissible (and consistent) heuristic, many graphs admit more than one shortest path of identical cost. In such cases, A* and Dijkstra may simply explore nodes in a different order (due to how ties are broken in their priority queues) and therefore reconstruct different-but equally optimal-routes